國立臺北科技大學
National Taipei University of Technology

資料結構
Data Structure

Lab 07

姓名:王顗程
學號: 114310215

Lab07-Ex1. (題目編號)
Exercise: Convert Infix Expression to Postfix Using a Stack Implemented with Linked List
Code

```
#include <iostream> // 引入標準輸入輸出功能
#include <string>   // 處理字串用
#include <cctype>   // 處理 isalnum() 判斷字母用
using namespace std;
// 節點結構,每個節點代表堆疊中的一個元素
struct Node {
        char data; // 存放資料(整數)
        Node* next; // 指向下一個節點的指標
};
// 定義堆疊類別
class Stack {
private:
        Node* top; // 指向堆疊頂端的指標
public:
        // 建構子:初始化堆疊為空
        Stack() {
                top = nullptr; // 空堆疊時 top 是 nullptr
        }
        // 推入元素到堆疊(stack.push(value))
        void push(char value) {
                Node* newNode = new Node(); // 建立新節點
                newNode->data = value; // 儲存資料
                newNode->next = top; // 新節點指向原本的頂端
                top = newNode; // 更新頂端為新節點
                //cout << newNode->data << " 已推入堆疊\n";
        }

        // 取出堆疊最上面的元素(stack.pop())
        char pop() {
                if (top == nullptr) { // 如果堆疊是空的
                        cout << "堆疊為空,無法取出元素。\n";
                        return '\0'; // 結束這次 pop()
                }
                char poppedValue = top->data; // 取得頂端元素的值
                Node* temp = top; // 用 temp 暫存目前頂端節點
                top = top->next; // 更新頂端為下一個節點

                delete temp; // 釋放記憶體,刪除原頂端節點
                //cout << "已取出元素: " << poppedValue << endl;
                return poppedValue;
        }

        char peek() {
```

```
            if (top == nullptr) return '\0';
            return top->data;
        }

        bool isEmpty() {
            return top == nullptr;
        }

        // 顯示目前堆疊的所有元素(由頂端往下列印)
        void display() {
            if (top == nullptr) { // 判斷堆疊是否為空
                cout << "堆疊為空。\n";
                return;
            }
            Node* current = top; // 從頂端開始往下走
            cout << "堆疊內容(頂端在前): ";
            while (current != nullptr) { // 走訪每個節點直到結尾
                cout << current->data << " "; // 印出節點資料
                current = current->next; // 移動到下一個節點
            }
            cout << endl;
        }
        // 解構子:程式結束時自動釋放所有節點記憶體
        ~Stack() {
            while (top != nullptr) {
                pop(); // 一個一個刪除直到堆疊為空
            }
        }
};

int getPriority(char op) {
        if (op == '*' || op == '/') {
            return 2; // 乘除優先
        }
        if (op == '+' || op == '-') {
            return 1; // 加減次之
        }
        return 0; // 括號 '(' 或堆疊為空時, 回傳最低分
}

string infixToPostfix(string infix) {
        Stack s;
        string postfix = "";

        for (int i = 0; i < infix.length(); i++) {
            char c = infix[i];

            // 1. 如果是數字或字母 (運算元), 直接加到結果字串
```

```
            if (isalnum(c)) {
                postfix += c;
            }
            // 2. 如果是左括號，無條件推入堆疊
            else if (c == '(') {
                s.push(c);
            }
            // 3. 如果是右括號，開始把堆疊的東西倒出來，直到遇到左括號
            else if (c == ')') {
                while (!s.isEmpty() && s.peek() != '(') {
                    postfix += s.pop();
                }
                s.pop(); // 最後左括號不放進結果
            }
            // 4. 如果是運算子 (+, -, *, /)
            else {
                // 如果堆疊頂端的優先權比我高(或一樣高)，就移到結果區
                while (!s.isEmpty() && getPriority(s.peek()) >= getPriority(c)) {
                    postfix += s.pop();
                }
                // 自己進堆疊
                s.push(c);
            }
        }

        // 最後，把堆疊裡剩下的全部倒出來
        while (!s.isEmpty()) {
            postfix += s.pop();
        }

        return postfix;
}


// 主程式
int main() {
        // 建立題目要求的三組測試案例
        string test1 = "(A+B)*D+E/(F+A*D)+C";
        string test2 = "A+B*(C*(D+E))";
        string test3 = "3+1*2+(4+5*6)/(9-7)";

        cout << "=== Infix to Postfix Conversion Test ===\n" << endl;

        // 測試第一組
        cout << "Test 1" << endl;
        cout << "Infix:   " << test1 << endl;
        cout << "Postfix: " << infixToPostfix(test1) << endl;
        cout << "-----------------------------------" << endl;
```

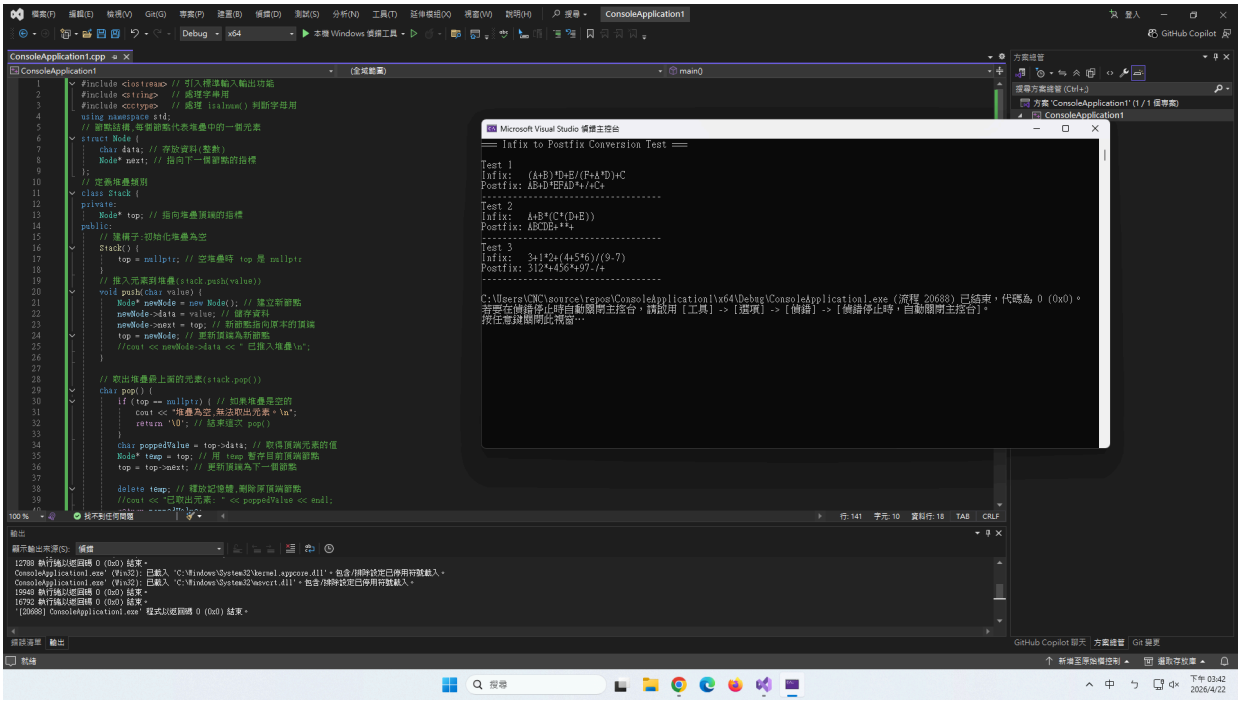
```
    // 測試第二組
    cout << "Test 2" << endl;
    cout << "Infix:   " << test2 << endl;
    cout << "Postfix: " << infixToPostfix(test2) << endl;
    cout << "-----------------------------------" << endl;

    // 測試第三組
    cout << "Test 3" << endl;
    cout << "Infix:   " << test3 << endl;
    cout << "Postfix: " << infixToPostfix(test3) << endl;
    cout << "-----------------------------------" << endl;

    return 0;
}
```

Lab01-Q1. (題目編號)

Code



Result

Discussion

Lab01-Q#. (題目編號)
Briefly describe the problem. (題目簡述)
Code

Paste your code here. Please use Pygments to highlight your code. (請使用Pygments傳換程式碼格式後再貼上)	Paste your code here. Please use Pygments to highlight your code. (請使用Pygments傳換程式碼格式後再貼上)

result		
(a)	(b)	(c)
	Show your result in tables or screenshots. (請使用表格或是截圖呈現結果。)	

Discussion
Discuss and conclude your results, or answer questions. (實驗討論、結論或回答問題。)